

EC8207– APPLIED BIG DATA ENGINEERING

Mini Project Assessment

The Smart City Traffic & Congestion System

PROJECT REPORT

Jayaweera S. T. EG/2020/3998

Gunathilake A. P. B. EG/2020/3947

Madhushan K. K. S. EG/2020/4356

Table of Contents

1.	Introduction.....	3
2.	System Overview & Architecture	4
3.	Tools & Technologies Justification	5
4.	Real-Time Stream Processing Logic.....	6
5.	Event Time Vs. Processing Time.....	6
6.	Batch Processing & Orchestration	7
7.	Analytics & Reporting Output	9
8.	Ethics & Data Governance.....	10
8.1	Privacy Implications	10
8.2	Data Governance Recommendations	10
8.3	Ethical Use	10
9.	Conclusion	11

1. Introduction

In this project, we develop a full-fledged smart city traffic pipeline with Lambda Architecture approach, wherein stream processing is combined with batch processing, with an aim to analyze both live and historical data. The system continuously analyzes the volume and velocity of vehicles in various city junctions in real time, detects critical traffic jam instances, and provides correct historical aggregation to enable effective deployment of police force on a daily basis.

The pipeline architecture includes:

- **Ingestion Layer:** Python traffic generator producing realistic IoT sensor telemetry.
- **Streaming Layer:** Apache Kafka for high-throughput event distribution; Apache Spark Structured Streaming for real-time congestion indexing and critical alert generation (speed < 10 km/h).
- **Real-Time Alerts & UI:** Immediate flagging of critical conditions to PostgreSQL, surfaced instantaneously on a Live JavaScript Dashboard.
- **Batch Processing Layer:** Apache Airflow orchestrating scheduled ETL jobs nightly to extract, aggregate, and warehouse traffic patterns.
- **Analytics Layer:** CSV-based data exports featuring Traffic Volume by Hour charts and Police Deployment Strategies.

This report documents the complete architecture, technology justifications, event-time handling semantics, batch orchestration, and the ethical data governance considerations required for a production-grade civic monitoring system.

2. System Overview & Architecture

The traffic monitoring pipeline implements a Lambda architecture combining real-time and batch processing paths:

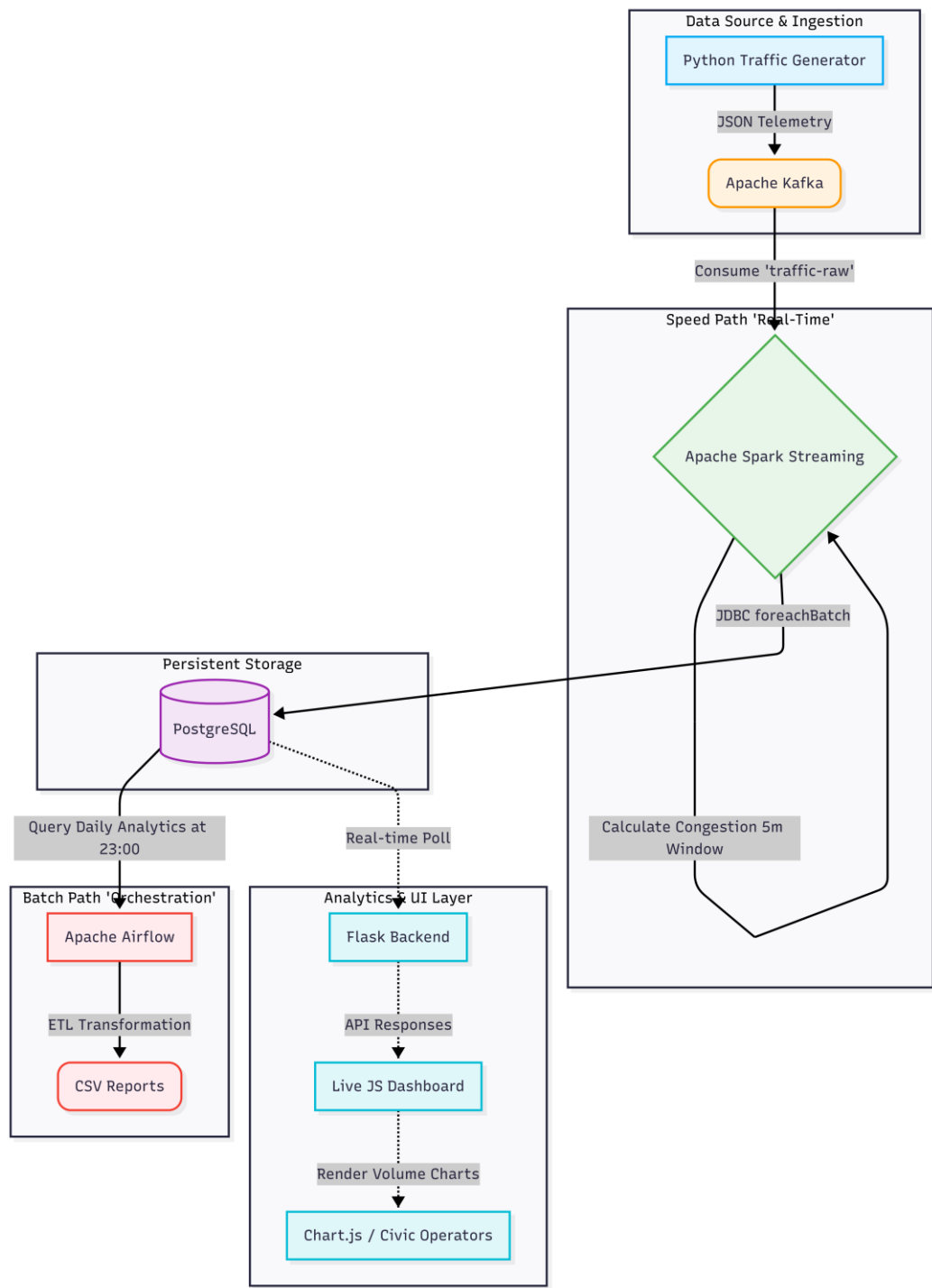


FIGURE 1: ARCHITECTURE DIAGRAM

Components of Architecture:

- **Data Source Layer:** Traffic generation tool mimicking sensors on 4 intersections, creating JSON documents containing `sensor_id`, `event_time`, `vehicle_count`, `avg_speed` attributes.
- **Event Ingestion Layer:** Kafka streams partition telemetry into the `traffic-raw` topic, allowing robust buffering and decoupling of input/output processes.
- **Stream Processing Layer (Speed Layer):** Kafka Streams performs real-time window-based aggregations and filtering based on event-time semantics and watermarking (maximum 10-minute lateness). It computes a Congestion Index over 5-minute tumbling windows and detects abrupt decreases in average speed.
- **Real-Time Data Storage Layer:** Streaming results are stored in PostgreSQL (`sensor_readings`, `critical_alerts`, and `congestion_windows`) using `foreachBatch` calls.
- **Live Dashboard Layer:** A Node/JS frontend powered by Flask serves dynamic web content, querying PostgreSQL to visualize real-time performance metrics, critical alerts, and interactive Chart.js charts plotting traffic flow versus time of day.
- **Batch Processing Layer (Batch Layer):** Apache Airflow schedules a daily job `traffic_report_dag.py` to run at 23:00, extracting data from the database, thus guaranteeing zero-loss analytics of the previous day's events.

3. Tools & Technologies Justification

- **Apache Kafka:** Facilitates massive input throughput of fine-grained IoT data. Partitioning handles load distribution, while persistent offsets ensure that no sensor information gets missed even when maintenance takes place downstream.
- **Apache Spark Structured Streaming:** Preferred over native Apache Storm/Flink because of the single interface used for both batch and stream processing. Windowing provides natively supported temporal aggregation operations (such as "5-minute block averages"), while microbatching ensures reliable exactly once semantics.
- **PostgreSQL:** Relational databases intrinsically support time series aggregation thanks to `TIMESTAMPTZ` columns. They are able to handle both massive streaming writes and read queries for dashboards simultaneously.
- **Apache Airflow:** Handles scheduling of daily reporting tasks. The automated DAG configuration and retry policies ensure reliable report generation even if there is a network outage.
- **JavaScript/Chart.js & Flask:** Instantly provides intuitive visualization for the local government to track traffic bursts and police deployment scores without writing any SQL.

4. Real-Time Stream Processing Logic

Two core processing schemas operate with event-time awareness:

- **Rule 1 - Critical Anomalies (Alerts):** Any sensor reporting an average speed below 10.0 km/h triggers an immediate critical alert route to PostgreSQL.

```
alerts = parsed.filter(col("avg_speed") < lit(10.0)).select(
    col("sensor_id"), col("event_time"), col("vehicle_count"), col("avg_speed")
)
```

- **Rule 2 - Congestion Indexation (Metrics):** Rolling 5-minute tumbling windows group traffic to generate a structural Congestion Index: $\max(0, (vc/150) * (1 - speed/60))$.

```
windowed = parsed \
    .withWatermark("event_time", "10 minutes") \
    .groupBy(col("sensor_id"), window(col("event_time"), "5 minutes")) \
    .agg(...)
```

5. Event Time Vs. Processing Time

- **Event Time:** When the sensor actually captured the traffic reading (timestamp payload).
- **Processing Time:** When Spark receives and computes the message.

This is important to consider when mapping out the traffic. For example, a huge traffic blockage happened in the location J1_KOLLUPITIYA at 17:00 UTC under heavy rains during monsoon season. A network problem at that particular area delayed the reporting of observations, and Spark received the report at 17:15 UTC.

If Spark uses processing time in this case, it would misreport the traffic blockage incident to happen at 17:15 when in reality, it was at 17:00 UTC. However, since Spark only uses event time (`.withWatermark("event_time", "10 minutes")`) for computations, this issue is already automatically handled.

6. Batch Processing & Orchestration

Scheduled batch jobs every night (23:00) extract and store traffic data. The Airflow DAG pipeline executes:

1. **Extract:** Queries traffic_db restricting purely to CURRENT_DATE.
2. **Transform:** Sorts anomalies chronically and organizes schema layouts.
3. **Load:** Iterates dynamically over sensor_readings, critical_alerts, and congestion_windows tables, saving data directly into persistent CSV files.

Nightly Outputs:

- reports/sensor_readings_YYYY-MM-DD.csv
- reports/congestion_windows_YYYY-MM-DD.csv
- reports/critical_alerts_YYYY-MM-DD.csv

TABLE 1: SENSOR DATA READINGS

id	sensor_id	event_time	processing_time	vehicle_count	avg_speed	ingested_at
496	J4_BORELLA	2026-05-09 12:29:39.32120 1+05:30	2026-05-09 12:29:40.004000+0 5:30	24	20.72	2026-05-09 12:29:39.322000 +05:30
466	J3_TOWN_HA LL	2026-05-09 12:29:39.32058 0+05:30	2026-05-09 12:29:40.004000+0 5:30	44	43.34	2026-05-09 12:29:39.321000 +05:30
476	J2_BAMBALA PITIYA	2026-05-09 12:29:39.31986 7+05:30	2026-05-09 12:29:40.004000+0 5:30	19	44.65	2026-05-09 12:29:39.320000 +05:30
495	J1_KOLLUPITI YA	2026-05-09 12:29:39.31935 5+05:30	2026-05-09 12:29:40.004000+0 5:30	50	58.27	2026-05-09 12:29:39.319000 +05:30
494	J4_BORELLA	2026-05-09 12:29:38.31751 7+05:30	2026-05-09 12:29:40.004000+0 5:30	41	42.63	2026-05-09 12:29:38.318000 +05:30
465	J3_TOWN_HA LL	2026-05-09 12:29:38.31674 8+05:30	2026-05-09 12:29:40.004000+0 5:30	64	19.64	2026-05-09 12:29:38.317000 +05:30
475	J2_BAMBALA PITIYA	2026-05-09 12:29:38.31590 0+05:30	2026-05-09 12:29:40.004000+0 5:30	11	26.48	2026-05-09 12:29:38.316000 +05:30

493	J1_KOLLUPITI YA	2026-05-09 12:29:38.31522 8+05:30	2026-05-09 12:29:40.004000+0 5:30	28	47.66	2026-05-09 12:29:38.315000 +05:30
492	J4_BORELLA	2026-05-09 12:29:37.31109 9+05:30	2026-05-09 12:29:40.004000+0 5:30	149	6.5	2026-05-09 12:29:37.312000 +05:30
464	J3_TOWN_HA LL	2026-05-09 12:29:37.30933 3+05:30	2026-05-09 12:29:40.004000+0 5:30	21	33.75	2026-05-09 12:29:37.310000 +05:30
474	J2_BAMBALA PITIYA	2026-05-09 12:29:37.30848 7+05:30	2026-05-09 12:29:40.004000+0 5:30	48	25.33	2026-05-09 12:29:37.309000 +05:30
491	J1_KOLLUPITI YA	2026-05-09 12:29:37.30789 5+05:30	2026-05-09 12:29:40.004000+0 5:30	39	23.84	2026-05-09 12:29:37.308000 +05:30
490	J4_BORELLA	2026-05-09 12:29:36.30547 2+05:30	2026-05-09 12:29:40.004000+0 5:30	49	27.58	2026-05-09 12:29:36.306000 +05:30
463	J3_TOWN_HA LL	2026-05-09 12:29:36.30408 9+05:30	2026-05-09 12:29:40.004000+0 5:30	48	26.89	2026-05-09 12:29:36.305000 +05:30
496	J4_BORELLA	2026-05-09 12:29:39.32120 1+05:30	2026-05-09 12:29:40.004000+0 5:30	24	20.72	2026-05-09 12:29:39.322000 +05:30

7. Analytics & Reporting Output

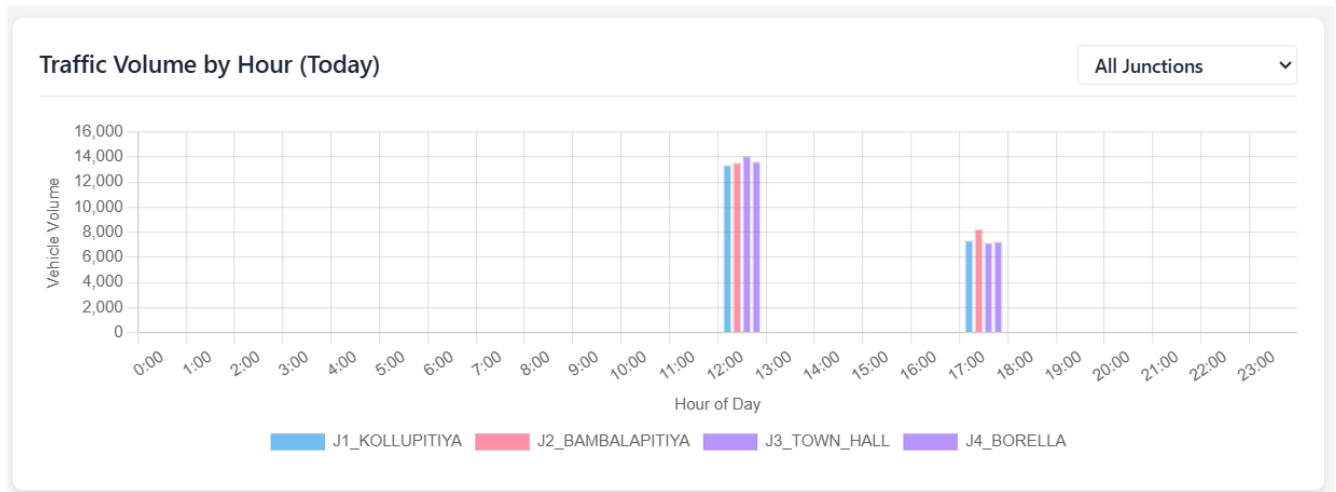


FIGURE 2: TRAFFIC VOLUME BY HOUR

The pipeline generates multi-tier analytic transparency:

Real-Time Dashboard (Visual Analytics):

- Live scrolling feed of intersection metrics.
- Blinking localized alerts highlighting exact junctions suffering sub-10km/h collapses.
- **Traffic Volume by Hour:** Bar chart (Chart.js) aggregating the real-time sum of vehicles mapped explicitly against the hour of the day.

Police Deployment Strategy (Reconciliation): A calculated daily score combining Peak Volume, Average Speed, and Critical Alerts thresholds.

- $\text{deployment_score} = \text{volume} * (1/\text{speed}) * (1 + \text{alerts})$
- Junctions scoring > 6.0 actively ping a visual **"DEPLOY POLICE"** notification, optimizing severely limited municipal policing resources reliably.

8. Ethics & Data Governance

8.1 Privacy Implications

The intent is that our intersection data will include aggregations at the macro-level (vehicle_count and avg_speed), thereby providing no personal information (no PII, such as ANPR and routing patterns). Nevertheless, there remains privacy concerns, especially as the data becomes granular, focusing on neighborhood egress points at precise times, which might allow for the determination of patterns. This means that data must be segmented and blurred by nature, especially concerning localized intersections near neighborhoods.

8.2 Data Governance Recommendations

Governance execution is key to the system. Firstly, there must be strict retention, where the raw telemetry data should be stripped off after 30 days and kept only in an analytical form. With regards to the situation in Sri Lanka, using the Personal Data Protection Act (2022), in case these devices were later equipped with optical/camera capabilities, the whole process should move behind the RBAC model, where incoming data should use the data masking technique and prevent any outgoing data from being transmitted outside the edge node.

8.3 Ethical Use

The automated report of police deployment creates an inherent complex ethical issue. Since enforcement scores are directly correlated to congestion levels, poor areas with deteriorating infrastructure leading to slow moving traffic may cause a huge spike in the number of “Police Deployments” only due to their systematic approach without any correlation to crime figures. This practice, in essence, leads to discrimination. Therefore, the thresholds established from such practices must only be used for traffic control/aide purposes and not for policing purposes at all.

9. Conclusion

Smart City Traffic Pipeline is able to effectively implement a sound lambda architecture to deal with real-time traffic telemetry data together with batch processing of the same data. This is because of the clear segregation between the Speed Path (Spark filtering Kafka inputs in real time with regard to sudden drops in speed) and the Batch Path (Airflow coordinating daily CSV export). In addition, the careful implementation of event-time semantics guarantees deterministic accuracy regardless of any network changes that may occur. All this comes with indispensable privacy criteria and biased documentation.